ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

LLM Benchmark project Report

Professors:

Michele Lombardi

Allegra De Filippo

Students:

Simone Migliaccio (matr.: 0001159303)

Simone Spezia (matr.: 0001167605)

Alberto Varini (matr.: 0001189363)

Contents

1 Introduction

The main goal of this project is to provide a general benchmark to evaluate different LLMs performances under different types of planning domains to better understand in advance what are the planning capabilities of a given LLM. In the following sections the main features of this projects are going to be shown.
Also a quick PDDL starting guide has been created to better gain the necessary knowledge about PDDL this specific language used to define planning problems.

2 General Structure

The root folder contains various sub folders, specifically:

- analysis
- archive
- Benchmark Framework
- results

2.1 analysis

This folder is used to gather all the datas useful for the production fo the report and the slides, contains the summary of the raw scores for the PDDL domains and the jupiter notebooks and the report.

2.2 archive

This folder contains all the material given us by the professor, specifically papers, a list of PDDL planning domains, two github repos given us as a reference and last a Quick PDDL guide written by us to help understand better the language.

2.3 Benchmark Framework structure

The folder Benchmark Framework is the core of all the work we've done. The main goal of this framework was to provide a easy to use and general testing platform to be employed when evaluating the performances of a LLM is needed, The following pages are going to be used to show the folders and the structure of the framework:

docs Contains a single file for the model preparation which i useful because the benchmark can run on a local PC or on an HPC system such as Leonardo. The important operational rule is that GPU benchmark jobs should not download large model weights. Prepare Hugging Face models first, then run benchmarks from local files.

evaluators Provides tools for to understand, validate and measure the capabilities of LLMs to solve planning problems

Leonardo script Contains specific scripts to launch and manage jobs on Leonardo HPC.

models Manages the communication with various LLMs, using "adapters" which creates a common interface for different backends like NVIDIA API.

outputs folder containing all the output from the LLMs, divided in 3 sub folders *raw, parsed, scored* to better separate respectively the raw output from the model,PDDL extracted and strctured plans and validation and metrics results.

prompts contains text files each one used to explain to the LLM the rules of the domain and how to manage the problem in order to solve it.

protocols defines different strategies for test execution, for example direct plan (which is one shot) or iteratoive repair.

reporting contains scripts for creation of teh plots used in the reports.

runner contains scripts useful to launch a singlke test or the whole suite.

scripts contains scripts for the computation of the numerical score assigned to each domain.

slurm logs Used to gather information for debug when the benchmark runs on Leonardo HPC

tasks PDDL problems and related instances selected for the runs, the instances are divided in 3 categories *easy, medium, hard*.

tests Used to run tests in preparation for the real run of the benchmark.

utils contains executable for the validator to be used during the run of benchmark framework

advanced planning evaluation.py it reads benchmark artifacts from the output folder and writes a model-centric JSON report in the folder results.

clear outputs is used to empty a folder during Leonardo tests

run benchmark is the file that runs the whole sequence of tests

The framework and its structure is organized in this way in order to allow teh user to change fundamental parameters such as the numbers of iterations easily and effectively and then run the benchmark again with the modified parameters.

2.4 results

3 PDDL Problems Classification

The first thing to do since we wanted a trustable benchmark and since a lot of PDDL models were already provided as reference material was to classify the problems using a numerical difficulty score. We used a numerical score because it is intuitively understandable by anyone who will ever want to compare the performance of LLMs using these models. Another classification implemented consists into separating problems in different categories based on the type of reasoning they request. To compute the scores the code can be found in the file `score_domains_complexity.py`

3.1 Difficulty Classification

3.1.1 Instances scores

The complexity of a single problem instance is calculated using a weighted logarithmic combination of four key metrics. The use of the logarithm prevents any single feature from dominating the score excessively.

The **Raw Score** is defined as:

$$Raw\_score = 0.45 \cdot \log(1 + A) + 0.20 \cdot \log(1 + G) + 0.20 \cdot \log(1 + N) + 0.15 \cdot T \tag{1}$$

Where:

- **A (Actions):** Number of feasible grounded actions. This is estimated by unifying action parameters with objects that satisfy static preconditions in the initial state.
- **G (Goals):** Total number of atomic conditions in the goal state.
- **N (Numeric Score):** A composite value representing numeric complexity:

  $$N = E_{num} + 2 \cdot P_{num} + 3 \cdot G_{num} + M \tag{2}$$

  where E_{num} are numeric effects, P_{num} numeric preconditions, G_{num} numeric goal conditions, and M is a binary flag for the presence of a metric.

- **T (Topology Score):** Evaluates the spatial/relational complexity based on predicates like *adj* or *connected*:

$$T = \log(1 + Nodes) + \log(1 + Edges) + Density \tag{3}$$

Obviously a different weight could be assigned to each different part of the computation of raw scores depending on the operator that is doing the tests in order to higlight more for example the number of action present in the instances and so on.
Finally, an **Instance Score (0-100)** is calculated by normalizing the raw score relative to the minimum and maximum scores found within the same domain. The results can be found in the folder `domains_complexity`.
Note that also some additional statistics for each domain are computed based on the results obtained from the instances, for example it could be of interest evaluating the mean and the median of the scores obtain form the instances of a single domain.

3.1.2 Domain scores

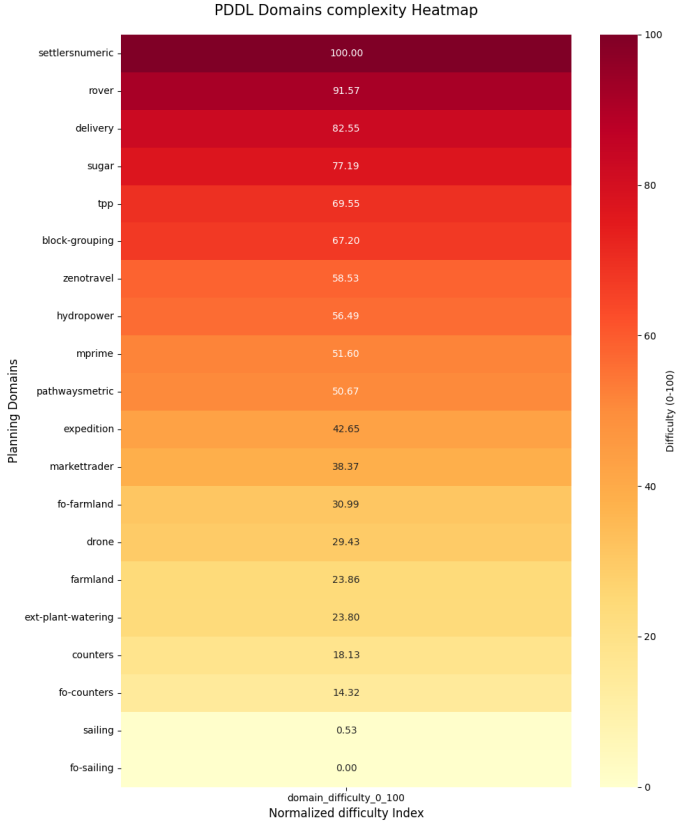
The overall difficulty of a domain is not just the average of its instances, but a reflection of its total state-space and constraint complexity. The **Domain Difficulty (0-100)** is computed by:

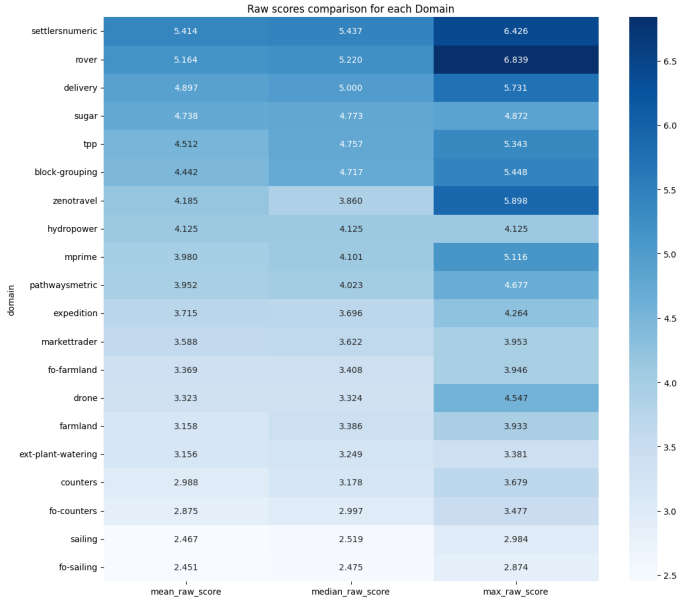
1. Summing the S_{raw} of all instances belonging to the domain to obtain a *Total Raw Score*.
2. Normalizing this total across all tested domains in the benchmark.

This approach ensures that a domain with many complex instances is ranked higher than one with few simple instances, note that obviously the easiest problems gets a domain difficulty equal to 0 (in this case is the "fo-sailing" problem) and the problem with the highest difficulty has domain difficulty equal to 100, (in our case the "settlersnumeric" problem).

3.1.3 Results obtained

In order to obtain a visual representation of the classified problems, we produced heatmaps for both the overall normalized domain scores and the raw scores.





3.2 Reasoning Classification

Here the classification based on the type of reasoning these problems require the LLM to employ is shown. We decided to split the reasoning types into 5 categories:

- **Spatial:** These problems are typically about objects that have to be moved in a 2D (or more dimensional) spaceframe.
- **Numerical Optimization:** These problems require from the LLM an extensive capacity

to deal with numerical optimization problems, such as managing complex cost functions and resource constraints.

- **Temporal Planning:** In this kind of problem it is required from the LLM to be able to reach the goal defining an order between objects having timing constraints and relations.
- **Logistic:** In this kind of problems it is required to reach the goal by planning some kind of goods transportation and agents coordination.
- **Logic:** These are abstract puzzles where the state space and actions represent symbolic or mathematical transformations rather than physical interactions. They test the model's ability to reason over purely formal rules without relying on world knowledge.

In the following table the classification of the 20 planning problems is shown, note that in this case all the work have been done by hand and that the problems in the table are ordered form hardest to easiest following the previous difficulty classification. Note also that the evaluation is based on how much the problem belong to the specific domain, for example in the problem "settlersnumeric" there is a dominant Numerical Optimization part hence the problem is going to be classified as a Numerical Optimization problem.

PDDL Problem	**Reasoning Category**
Settlersnumeric	Numerical Optimization
Rover	Logistic
Delivery	Logistic
Sugar	Numerical Optimization
Tpp	Numerical Optimization
Block-Grouping	Spatial
Zenotravel	Logistic
Hydropower	Temporal Planning
Mprime	Logic
Pathwaysmetric	Numerical Optimization
Expedition	Logistic
Markettrader	Numerical Optimization
Fo-farmland	Numerical Optimization
Drone	Spatial
Farmland	Numerical Optimization
Ext-Plant-Watering	Spatial
Counters	Numerical Optimization
Fo-Counters	Numerical Optimization
Sailing	Spatial
Fo-Sailing	Spatial

Table 1: Reasoning Category Classification Tab

3.3 Problems chosen

In order to chose the correct problem to be solved by LLMs we decided to pick 3 sets composed by 3 problems each. The sets are composed by 3 problems with different reasoning categories and specifically from the categories Numericaal Optimization, Spatial and Logistic. Problems related to Temporal Planning and Logic were not chosen since just one problem for each type was present; we suggest, in order to study the capabilities of LLMs in these domains, to use a PDDL problenms dataset specifically containing these kind of problems.
In order to better understand how we picked the problems the following tables are shown, the elements are ordered with decreasing difficulty.

Numerical Optimization
Settlersnumeric
Sugar
Tpp
Pathwaysmetric
Markettrader
Fo-farmland
Farmland
Counters
Fo-Counters

Spatial
Block-Grouping
Drone
Ext-Plant-Watering
Sailing
Fo-Sailing

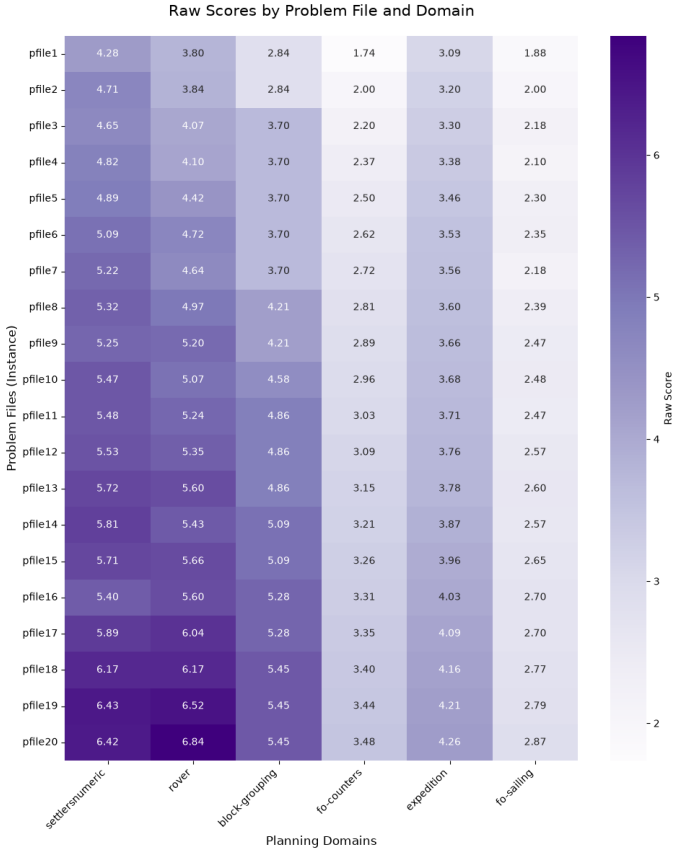
Logistic
Rover
Delivery
Zenotravel
Expedition

Table 2: Reasoning Categories Selected for Benchmark

In order to obtain a more clear output from the LLM we decided to select two differents sets of three problems each and specifically:

- **Hard Set:** composed by *settlersnumeric, block-grouping, rover*
- **Easy Set:** composed by *fo counters, fo sailing, expedition*

It is clearly visible that we picked respectively the most three difficult problems and the three easiest ones. This has been done to ensure also in the outputs a visible difference in the metrics when evalauting the outputs.
Additionally it can be seen from the following heatmap that the instances of these domains have increasing difficulty:



It is clearly visible both the difference in terms of difficulty between the Hard set and the Easy set and the increasing difficulty related to the pfiles; because of this last consideration we applied an additional, partition to divide the pfiles in three different categories and specifically:

- Easy (composed by pfiles from 1 to 4)
- Medium (composed by pfiles from 8 to 11)
- Hard (composed by pfiles from 15 to 18)

Also it is worth noticing that we did not use all the instances available for each selected domain but we decided to choose a small batch (4 instances) of instances representing the easy, medium and hard categories.

4 Configured LLMs

The benchmark framework was designed to support two execution backends: Leonardo HPC by CINECA and the NVIDIA API. Leonardo was used for local execution of Hugging Face models, while the NVIDIA API provided remote access to additional models without requiring every checkpoint to be installed and hosted locally.

4.1 Execution Backends

4.1.1 Leonardo HPC

Leonardo HPC was the main platform considered for local model execution. The models were loaded through the Hugging Face backend, allowing direct control over the execution environment, GPU resources, precision, prompt formatting, and generation parameters.

The selection of locally executable models was supported by whichLLM, a repository used to estimate which LLMs can run on a given hardware configuration and to compare models by release recency. This was useful to identify models that were realistic candidates for the available Leonardo resources before running the benchmark.

4.1.2 NVIDIA API

The NVIDIA API was used as a remote inference backend. This reduced the setup effort required to test recent open-weight models and made it possible to include models that would have been less convenient to install or run locally.

The benchmark pipeline remained the same across both backends: prompts, parsing, validation, and scoring followed the same workflow, while only the model adapter changed. The NVIDIA API backend also introduced practical constraints, such as rate limits, temporary service availability, and model-specific API key requirements.

4.2 Selection Criteria

The models were selected according to three practical criteria:

- **Hardware feasibility:** local models had to be compatible with the available Leonardo environment, using whichLLM as a practical reference for hardware requirements.
- **Recency:** priority was given to recent LLM families, using whichLLM as a reference to compare model release dates.
- **Model diversity:** the benchmark included models from different families to avoid evaluating a single architectural lineage.

Using both backends made the framework more flexible than a Leonardo-only setup. Leonardo provided controlled local execution, while the NVIDIA API allowed the benchmark to cover additional hosted models through the same interface.

4.3 Configured Models

The implemented model set includes the models configured for experimentation in the framework. Since the benchmark runs were still being refined, this section describes the configured model pool rather than treating any single run as the definitive final result.

Backend	Execution mode	Configured models
Leonardo HPC	Local HF	Gemma 4 31B, Phi-4, Nemotron Nano, Qwen 3.6
NVIDIA API	Remote hosted	Phi-4 Mini, GPT-OSS 120B, Kimi K2.6, Llama 3.3 70B, Nemotron Nano, Qwen 3.5

Table 3: LLM models configured in the benchmark framework.

5 Benchmark output Parsing and Output Structure

DA RIEMPIRE

6 Output Validation Using VAL

DA RIEMPIRE

7 LLMs Reasoning Capabilities Evaluation

7.1 Pipeline and Data flow

How Raw / Parsed / Scored become one DataFrame , which Classes hold the state the metrics need and why the load → row → aggregate → plot → report stages. This section maps the analysis script `advanced_planning_evaluation_sp.py` onto the rest of the Repo , showing how the three on-disk artifact layers Raw / Parsed / Scored. become DataFrames , which data classes exist and why and how the data-manipulation stage hands off to metric computation and visualization.

7.1.1 Pipeline Generic description

tcolorboxskins,breakable

tikz

bmkeybox[1]enhanced,breakable,colback=accent!4,colframe=accent!55, coltitle=ink,fonttitle=**,title=1, left=8pt,right=8pt,top=6pt,bottom=6pt,boxrule=0.4pt, borderline west=2.5pt0ptaccent** bmcavbox[1]enhanced,breakable, coltitle=ink,fonttitle=**,title=1, left=8pt,right=8pt,top=6pt,bottom=6pt,boxrule=0.4pt, borderline west=2.5pt0ptamber** bmformulaboxcolback=codebg,colframe=linecol,boxrule=0.4pt, left=10pt,right=10pt,top=7pt,bottom=7pt

3	**1**	**8**	**27**	**7**
artifact layers	canonical DataFrame	aggregate tables	registered plots	capability profiles

7.1.2 The pipeline at a glance

Everything funnels into one DataFrame, `df_metrics`, the single source of truth for every `groupby` and every plot. Solid (teal) arrows are the data path; dashed (grey) arrows inject the PDDL ground truth needed to score plans (Figure **??**).

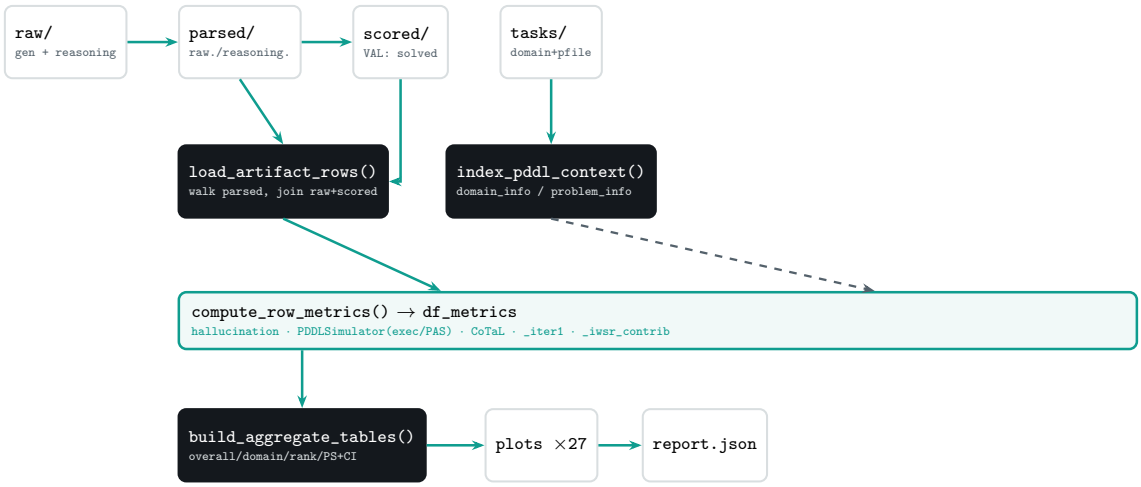


Figure 1: The analysis pipeline. One DataFrame, `df_metrics`, is the single source of truth; PDDL ground truth (dashed) is injected to score plans.

7.1.3 How a row is built: the three inputs to `df_metrics`

The canonical table is produced by `compute_row_metrics()` . It does *not* read the filesystem and it does *not* compute ground truth — both happen before it. By the time the row loop runs, three things already exist, each with a structurally different role.

What `df_metrics` literally is `df_metrics = df_raw.copy()` horizontally concatenated with three blocks of computed columns — hallucination, precondition/executability (from `PDDLSimulator`), and CoTaL — plus two derived columns `_iter1` and `_iwsr_contrib` . So input 1 supplies the

#	Input	Produced by	Role in the row loop
1	`df_raw`	`load_artifact_rows()`	The skeleton: one row per instance with the artifact-derived columns and the keys used to look up ground truth. It is `.copy()`-ed, then *widened* with computed columns.
2	`domain_info` / `problem_info`	`index_pddl_context()`	The ground truth: two dict lookups (`d_info`, `p_info`) consumed per row to score the plan. *Not* stored as columns — they are the reference the metric functions compare against.
3	`warnings_out`	threaded list via `add_warning()`	The diagnostic side-channel: a by-reference list accumulating structured records whenever an input is missing or malformed. Never part of `df_metrics`; embedded separately in the report.

columns you keep; input 2 is *consumed* to manufacture the new columns and then discarded; input 3 is a parallel log, not a column. The "three components" mental model is right, but only the first is literally part of the frame.

`df_raw` — the artifact skeleton `load_artifact_rows()` walks the canonical path `parsed/<run>/<model>/<protocol>/<domain>/<difficulty>/<instance>.json` and, for each instance, joins the matching `scored/` data. It emits exactly *one row per problem instance*, collapsing the retry loop by keeping the last non-empty plan as the representative `_actions` while preserving the full attempt lists as private columns.

```
# public  -- identity + outcome
Model, Domain, Problem, Difficulty, Protocol, Run_id
Valid (= scored["solved"]), Length, Iterations (= scored["iterations_used"]), Chain_of_Thought
# private -- consumed downstream, dropped from the report
_actions          # representative plan (last non-empty attempt)
_cot_text         # reasoning trace text
_parsed_attempts  _scored_attempts  _raw_attempts   # full per-iteration lists
_file_path  parsed_file_path
```

The keys that matter for the next input are `Domain`, `Difficulty` and `Problem` — together the exact triple used to retrieve per-instance ground truth. The `Chain_of_Thought` flag is resolved from three sources (protocol YAML, presence of reasoning text, presence of reasoning actions) so CoT-bearing runs are never mislabelled.

Why one row per instance, not per attempt Keeping the grain at the *instance* level means every aggregate (success rate, hallucination, PS) is automatically an average over comparable units. The iteration structure is not lost — it survives in the private `_*_attempts` columns and the `Iterations` count — but it is opt-in, read only by metrics that need it (FASR, IWSR, the iteration profile, CoTaL).

The PDDL parsing layer — `d_info` and `p_info` Plans can only be scored against the formal task they were generated for, so the framework parses the PDDL itself. Two small, deliberately permissive regex parsers do this — a syntax error emits a warning, never aborts the run. They answer two different questions and return two differently-shaped dicts.

`parse_domain_pddl(domain_path)` reads one `domain.pddl` (the schema shared by every instance in a domain) and returns the legal vocabulary plus per-action templates:

```
parse_domain_pddl(...) -> {
  "action_names": set[str],   # every legal action head -- the hallucination whitelist
  "predicates":   set[str],   # legal predicate names
  "functions":    set[str],   # numeric fluent names -- what is numeric, not propositional
  "schemas": {                # one entry PER action, the simulator's instruction sheet
```

```
    action_name: {
       "params":   [str, ...],   # ordered ?param variable names, for grounding
       "prec_raw": str,          # raw :precondition s-expression text (unparsed)
       "eff_raw":  str,          # raw :effect s-expression text (unparsed)
    }, ...
  },
  "path": str | None,         # None when the file was missing (empty stub)
}
```

Each field has a single downstream consumer: `action_names` is the whitelist for action hallucination; `functions` tells the simulator which heads are numeric fluents; and `schemas` is what the simulator grounds — `params` maps the action's arguments onto the variables inside the raw `prec_raw` / `eff_raw` strings so preconditions can be checked and effects applied.

`parse_problem_pddl(problem_path)` reads one instance file (its own object set and starting state) and returns:

```
parse_problem_pddl(...) -> {
  "objects":      set[str],               # legal argument tokens -- object-hallucination
    whitelist
  "init_atoms":   set[tuple[str, ...]],   # initial true propositions, e.g. ("at","truck1","depot
    ")
  "init_numeric": dict[tuple, float],     # initial fluent values, e.g. ("fuel","truck1") -> 50.0
  "path":       str,
  "difficulty": str,                      # parent folder name
}
```

Object hallucination uses `objects`; the simulator seeds its world from `init_atoms` and `init_numeric`. Action- vs object-hallucination are kept as separate signals on purpose — a model can know the legal action names yet still invent objects, a different and shallower grounding failure.

`index_pddl_context(tasks_dir)` calls those two parsers exactly once each per file, walking `tasks/` a single time, and packs the results into two lookups whose key shapes mirror the granularity of what they hold:

```
domain_info  : dict[str, dict]                 # key = domain_name  (schema is shared)
problem_info : dict[tuple[str,str,str], dict]  # key = (domain, difficulty, instance)
```

This is the "parse once, consume many" design: every PDDL file is read once regardless of how many rows or retries reference it. In the row loop the retrieval is then two O(1) dict gets:

```
d_info = domain_info.get(domain, {})                          # by domain
p_info = problem_info.get((domain, difficulty, instance), {})  # by full triple
pddl_ok = bool(d_info.get("action_names")) and bool(p_info.get("objects"))
```

A missing domain and a missing problem fail differently When `domain.pddl` is absent, `index_pddl_context` still stores a populated-but-empty stub under the domain key (empty `action_names`, empty `schemas`, `path=None`). When a problem file is absent, no entry is created and the row's `.get(..., {})` returns a bare `{}`. Both end up empty, but only the domain case leaves a record — and an empty `action_names` set means *every* emitted action looks illegal, silently depressing that domain's grounding numbers. The only trace is a warning, which is exactly what input 3 is for.

`warnings_out` — the diagnostic accumulator `warnings_out` is a single `list[dict]` created once at the top of the analysis run and threaded by reference into every stage that can encounter a malformed or missing input — `index_pddl_context`, both PDDL parsers, and

`compute_row_metrics`. Each stage appends through one helper:

```
def add_warning(warnings_out, warning_type, message, **extra):
    warnings_out.append({"type": warning_type, "message": message, **extra})
    # **extra carries locating context: run_id, domain, difficulty, problem, path
```

Its purpose is to replace scattered `print` / `logging` calls with one structured, serialisable trace that travels with the results: the full list is embedded in `report.json` under the `warnings` key, so a notebook or CI job can filter by `type` and locate the offending artifact without re-running anything. Typical entries are `missing_tasks_dir`, `missing_domain_pddl`, `missing_pddl_context`, `domain_parser_error`, `problem_parser_error`. Rows with incomplete context are *not* dropped — they still get NaN metric values — so the instance count stays honest while the warning records why those NaNs exist.

One diagnostic path bypasses `warnings_out` The simulator is the exception. If `compute_precondition_metrics` throws, it falls back to a NaN result and reports through Python's stdlib `warnings.warn("[simulator] ...")` — *not* through `add_warning`. So a simulator crash does not appear in the structured `warnings` list in the JSON report; it surfaces only on stderr. Worth unifying if the report is to be the single audit log.

7.1.4 The merge: three inputs into one widened frame

`compute_row_metrics` iterates `df_raw` once. For each row it resolves `d_info` / `p_info` (input 2), runs the three metric families against the row's `_actions` and attempt lists, and emits any incompleteness into `warnings_out` (input 3). The per-family results are collected and concatenated columnwise back onto the copied skeleton.

```
df = df_raw.copy()
for _, row in df.iterrows():
    d_info  = domain_info.get(row["Domain"], {})
    p_info  = problem_info.get((row["Domain"], row["Difficulty"], row["Problem"]), {})
    if not (d_info.get("action_names") and p_info.get("objects")):
        add_warning(warnings_out, "missing_pddl_context", ...)        # input 3
    halluc_rows.append(      compute_hallucination_metrics(actions, d_info, p_info))
    precondition_rows.append(compute_precondition_metrics(actions, d_info, p_info))  #
     PDDLSimulator
    cot_rows.append(         ... compute_cot_alignment_for_attempt(...) ...)

df_metrics = pd.concat([df, halluc_df, precondition_df, cot_df], axis=1)
df_metrics["_iter1"]        = (df.Valid == True) & (df.Iterations == 1)   # -> FASR
df_metrics["_iwsr_contrib"] = 1/Iterations if Valid else 0                # -> IWSR
```

The result, `df_metrics`, is the single source of truth every `groupby` in `build_aggregate_tables` and every plot reads from. Note the grain: the hallucination and executability columns describe the *representative* (last non-empty) plan, while the CoTaL and efficiency signals reach back into the preserved attempt lists.

7.1.5 Data classes that make the metrics possible

Two classes carry state that a flat function could not — introduced here, with the simulator expanded in full in Section **??**.

PDDLSimulator (in the analysis script). A forward STRIPS + simple-numeric simulator written in pure Python so the analysis layer needs no VAL subprocess. Instantiated *once per (problem, plan)* inside `compute_precondition_metrics`. Its decisive feature is `prop_hist`: a list of `frozenset` snapshots of the propositional state after every applied step. That history is

what lets the framework, at the first failing action, ask *"was this precondition ever true earlier?"* — the question separating a sequencing error (true before, deleted too early) from a state fabrication (never true). Without the per-step history there is no PAS and no temporal distance.

RunMetrics (`evaluators/metrics.py`, `@dataclass`). The validator-side metric bundle written at scoring time: `validity_at_1`, `validity_at_k`, `repair_success`, `iterations_to_valid`, `plan_length`, `error_type`, `hit_iteration_limit`. The analysis script re-derives most of these from the scored JSON, but this dataclass is the contract guaranteeing every model is scored on the same basis, independent of backend.

The error vocabulary is fixed in `evaluators/error_taxonomy.py` as an `ErrorType` enum, partitioned into `LOGICAL_ERROR_TYPES` (empty plan, syntax, unknown action, invalid precondition, unsatisfied goal) versus `TECHNICAL_ERROR_TYPES` (parse error, timeout, validator crash/unavailable). That split keeps infrastructure failures from being charged against a model's planning score.

7.1.6 `PDDLSimulator` in depth

Almost every execution-side discriminator — executability ratio, the sequencing-vs-fabrication split, PAS, and mean temporal distance — is produced by one small class. It is a **forward STRIPS + simple-numeric simulator written in pure Python**, deliberately so: the analysis layer can re-derive these signals without spawning a VAL subprocess or shipping a compiled binary. It reproduces just enough of the validator's logic for the propositional and numeric-inequality fragments the benchmark uses, trading completeness for portability.

What it is for. The class does three things: it **holds a world state** (a set of true propositions plus a dict of numeric fluents), it **walks the plan one action at a time** checking each action's preconditions before applying its effects, and — its decisive feature — it **saves a frozen snapshot of the state after every applied step**. That snapshot history is what makes the precondition-awareness metrics computable. It is the basis for four metrics: executability ratio (how far the plan runs before the first break), the sequencing-error / state-fabrication counts, PAS (their ratio), and mean temporal distance. It targets *forward*, *propositional* and *simple-numeric* domains — STRIPS add/delete effects plus `increase` / `decrease` fluents and $\geq \leq > <$ comparisons — and is invoked exactly once per `(problem, plan)` pair.

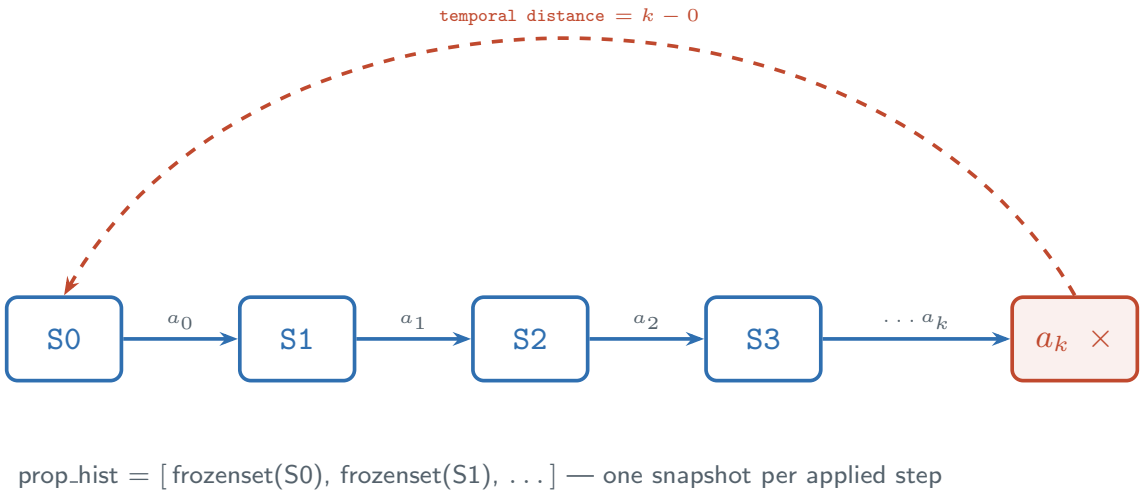


Figure 2: State evolution. At the first failing action a_k, each unmet precondition atom is traced back through `prop_hist`: found earlier → sequencing error (record $k-$ last as temporal distance); never found → state fabrication.

State, snapshots, and the step loop. The constructor seeds the world from the parsed problem and takes the first snapshot before any action runs; `prop_hist` then grows by one frozen set per applied step.

```
def __init__(self, d_info, p_info):
    self.prop      = set(p_info.get("init_atoms", set()))   # true propositions
    self.num       = dict(p_info.get("init_numeric", {}))   # numeric fluents
    self.prop_hist = [frozenset(self.prop)]                 # snapshot S0 = initial state
    self.step      = 0
```

For each action, `compute_precondition_metrics` grounds the schema, checks preconditions, and — only if they hold — applies the effects and appends a new snapshot. The walk **stops at the first failing action** (a `break`), which makes "how far did it get" a meaningful prefix length.

```
for step, action_str in enumerate(actions):
    name, args = parse_action(action_str)
    schema = schemas.get(name)
    if name is None or schema is None:        # unparseable / unknown action: skipped
        continue
    if len(schema["params"]) != len(args):    # wrong arity -> fabrication, stop
        first_failure = step; fab_errors += 1; break
    ok, failed_prop, failed_num = sim.check_preconditions(schema["prec_raw"], params, args)
    if not ok:
        first_failure = step
        for atom in failed_prop:
            last = sim.last_step_atom_true(atom)         # search the snapshot history
            if last >= 0: seq_errors += 1; temporal_distances.append(step - last)
            else:         fab_errors += 1
        fab_errors += len(failed_num); break
    sim.apply_effects(schema["eff_raw"], params, args)   # mutate state + append snapshot
```

The classification logic, atom by atom. At the failing action, each unsatisfied propositional precondition is the unit of analysis. `last_step_atom_true(atom)` scans `prop_hist` newest-to-oldest and returns the latest snapshot index at which the atom held — or `-1` if it never did.

```
def last_step_atom_true(self, atom):
    for index in range(len(self.prop_hist) - 1, -1, -1):
        if atom in self.prop_hist[index]:
            return index
    return -1
```

That single lookup is the whole basis of the sequencing/fabrication distinction. If the atom was true at some earlier snapshot, the model had the prerequisite and then ordered actions so as to destroy it — a **sequencing error**, and the gap step − last is recorded as a temporal distance. If the atom was never true, the model required a world state that never existed — a **state fabrication**. Effects are applied in STRIPS order (deletes before adds), so the snapshots faithfully reflect what each action left behind. The metrics fall straight out of these counts:

```
executability_ratio = first_failure / plan_len
PAS = sequencing_errors / (sequencing_errors + state_fabrications)
mean_temporal_distance = mean(step - last_true_step for each sequencing error)
```

The counts describe the first failure, not the whole plan Because the loop `break`s at the first failing action, the sequencing/fabrication counts and the temporal distances come from that *one* action's unmet preconditions — PAS is a property of where the plan first breaks, not a tally over the entire sequence. This is intended (it localises the failure), but it means a plan can accrue at most one failure event in these columns.

Unknown actions are skipped, which inflates executability An action whose name is not in the schema is `continue`d over — it neither advances the state nor counts as a failure here (name-

level hallucination is scored separately). So a plan made entirely of hallucinated action names never triggers a precondition failure and records `executability_ratio = 1.0`. Always read ER alongside the hallucination columns. Relatedly, because skipped actions advance the plan index but not the snapshot index, temporal distance is measured in generated-plan positions relative to the snapshot where the atom last held — the two clocks coincide only when every prior action was a known, applied schema.

7.1.7 From data to metrics to pictures — why each stage exists

The architecture separates concerns on purpose, so each is independently testable.

Stage	Function	Purpose / why it is separate
Load	`load_artifact_rows`	Decouples the filesystem walk from metric logic; everything below receives a uniform table.
Row metrics	`compute_row_metrics`	One pass attaches every per-instance metric; private `_actions` / `_cot_text` stay alive only here.
Aggregate	`build_aggregate_tables`	All `groupby` logic in one place so the JSON report can embed tables whether or not plots are drawn.
Visualise	`maybe_make_plots`	Reads only the aggregate tables; registering each figure embeds its path + description into the report.
Assemble	`build_model_payloads`	Re-keys everything by *model* so a consumer can pull one model in O(1), and attaches the capability-profile classification.

A real artefact to know about Because `compute_row_metrics` keeps the last non-empty plan as the representative for the instance-level hallucination/executability columns, those columns describe the *final* attempt. Metrics that must see all attempts (FASR, IWSR, CoTaL per-iteration, the iteration profile) read the preserved `_*_attempts` lists instead. When reading a plot, always check which grain it is on — this is spelled out per metric in the following sections.

7.2 Metrics scientific foundation

7.3 Metric Families

7.3.1 Planning Efficiency

7.3.2 Execution and Reasoning

7.3.3 Grounding and Discriminatory

7.3.4 PS (Composite Planning Score)

7.4 Cross-Domain and Cross-LLM comparison

8 Conclusions

DA RIEMPIRE